Cairo University
Faculty of Engineering

Assignment 3

Signal Space Representation & Gram-Schmidt Orthogonalization

Submitted by:

Hossam Elden Mohamed Ahmed
Student ID: 9230331

Course: ELC3253B – Digital Communications
3rd Year – Spring 2025–2026

1. Introduction

This report presents the implementation and analysis of signal space representation using Gram-Schmidt orthogonalization. Two signals s_1 (t) and s_2 (t) (defined over [0,1] sec) are studied in signal space, and the effect of Additive White Gaussian Noise (AWGN) on their representations is investigated.

2. Signal Definitions

The two signals used in this assignment, taken from Figure 1.1 of the assignment sheet, are:

- s_1 (t): A rectangular pulse of amplitude 1 over the interval [0, 1] sec.
- s_2 (t): A signal with amplitude −1 over [0, 0.25) sec and amplitude +1 over [0.25, 1] sec.

Both signals have a duration of T = 1 second and are sampled using N = 1000 samples with a time step dt = 1/1000 sec.

3. Part 1.1 – Gram-Schmidt Orthogonalization

3.1 Method

The Gram-Schmidt procedure constructs an orthonormal basis from the two input signals:

- φ_1 (t) is obtained by normalising s_1 (t): $\varphi_1 = s_1 / \|s_1\|$, where $\|s_1\| = \sqrt{(\int s_1{}^2 dt)}$.
- A residual g_2 is formed by subtracting the projection of s_2 onto φ_1 : $g_2 = s_2 - \langle s_2, \varphi_1 \rangle \varphi_1$.
- φ_2 is obtained by normalising g_2 . If $\|g_2\| \approx 0$ (linearly dependent), φ_2 is set to the zero vector.

3.2 Results

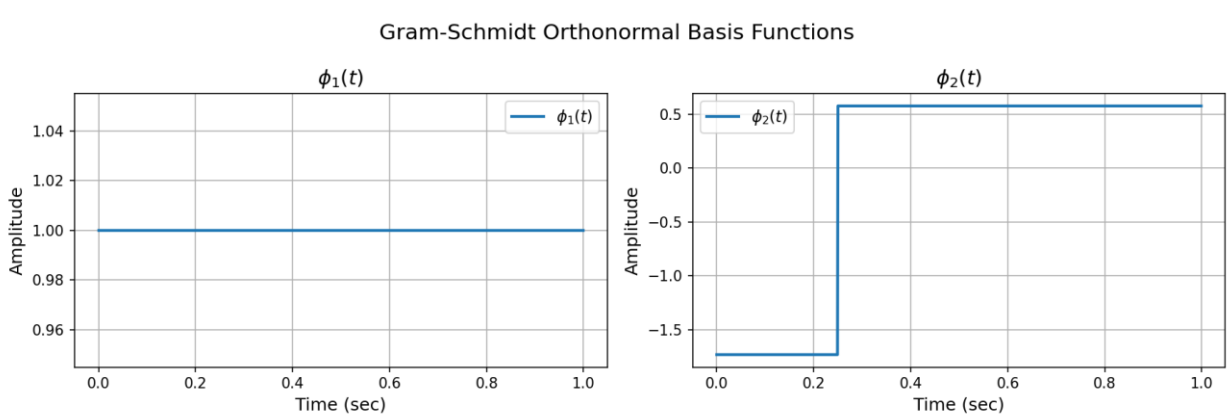
Since s_1 (t) = 1 over [0,1] with energy E = 1 J, the first basis function is:

φ_1 (t) = s_1 (t) / √1 = 1 for t ∈ [0, 1]

After removing the φ_1 component from s_2 , the normalised residual gives:

φ_2 (t) = −1.7321 for t ∈ [0, 0.25), +0.5774 for t ∈ [0.25, 1]

Figure 1: Gram-Schmidt Orthonormal Basis Functions φ_1 (t) (left) and φ_2 (t) (right)



3.3 Theoretical Derivation

Step 1 — Energy of s_1 (t):

$E_1 = \int_0^1 s_1^2(t)\,dt = \int_0^1 (1)^2\,dt =$ **1 J**

Step 2 — First basis function φ_1 (t):

$\varphi_1(t) = s_1(t) / \sqrt{E_1} = 1 / \sqrt{1} =$ **1, t ∈ [0, 1]**

Step 3 — Projection coefficient c_{21} :

$c_{21} = \int_0^1 s_2(t)\cdot\varphi_1(t)\,dt = \int_0^{0.25} (-1)(1)\,dt + \int_{0.25}^1 (+1)(1)\,dt$

$= (-1)(0.25) + (+1)(0.75) = -0.25 + 0.75 =$ **0.5**

Step 4 — Residual g_2 (t):

$g_2(t) = s_2(t) - c_{21}\cdot\varphi_1(t)$

$g_2(t) = -1 - 0.5(1) =$ **−1.5**, $t \in [0, 0.25)$

$g_2(t) = +1 - 0.5(1) =$ **+0.5**, $t \in [0.25, 1]$

Step 5 — Energy of g_2 and normalization:

$E_2 = \int_0^{0.25} (-1.5)^2\,dt + \int_{0.25}^1 (0.5)^2\,dt = (2.25)(0.25) + (0.25)(0.75) = 0.5625 + 0.1875 =$ **0.75**

$\varphi_2(t) = g_2(t) / \sqrt{0.75}$

$\varphi_2(t) = -1.5 / \sqrt{0.75} \approx$ **−1.7321**, $t \in [0, 0.25)$

$\varphi_2(t) = +0.5 / \sqrt{0.75} \approx$ **+0.5774**, $t \in [0.25, 1]$

Verification: $\int \varphi_1(t)\varphi_2(t)\,dt = (1)(-1.7321)(0.25) + (1)(0.5774)(0.75) = -0.4330 + 0.4330 =$ **0** ✓

4. Part 1.2 – Signal Space Representation

4.1 Method

The signal space coordinates (v_1 , v_2) of a signal s(t) are computed by projecting it onto the basis functions via inner products (numerical integration):

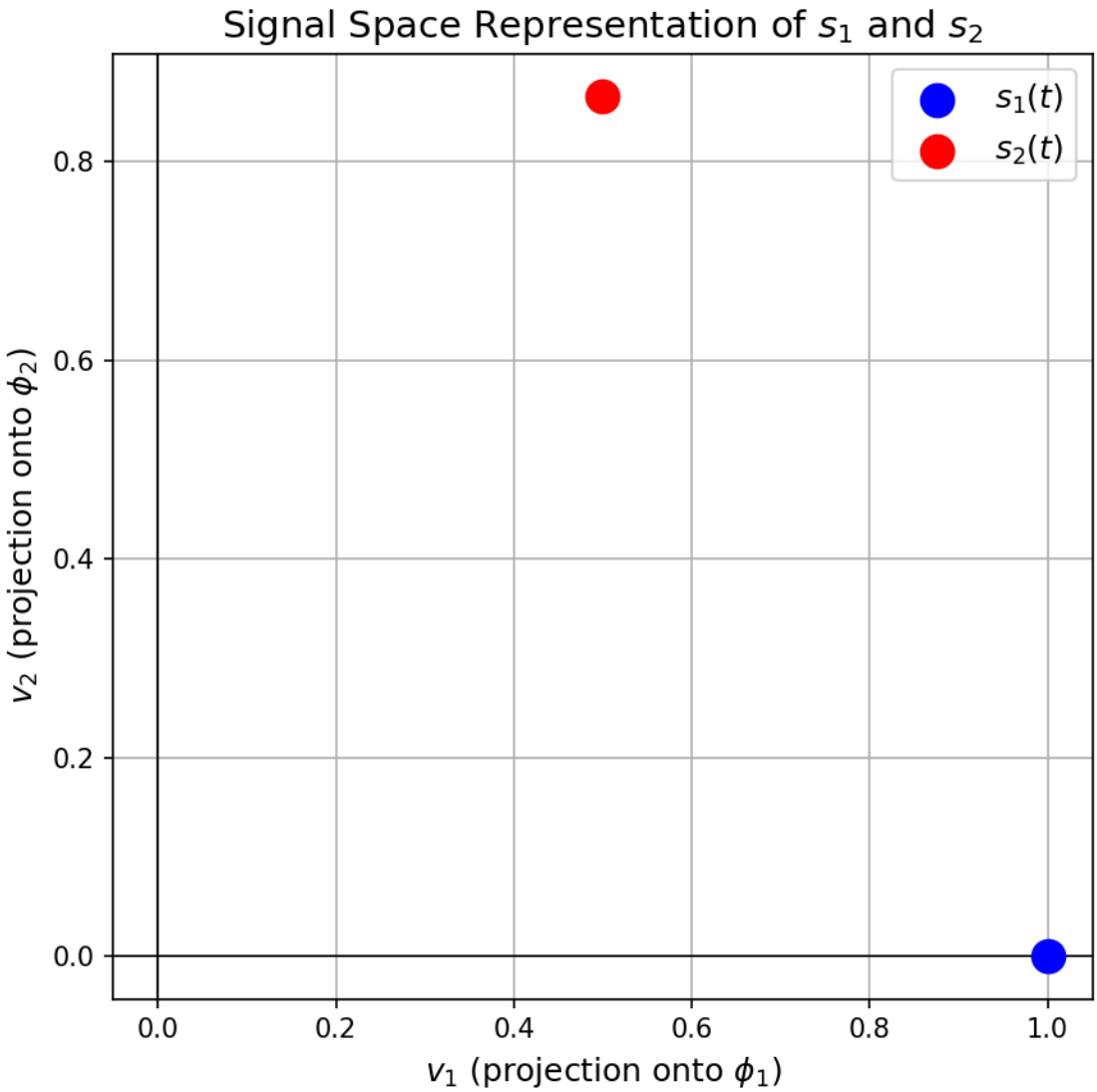
$$\mathbf{v_1 = \int s(t)\, \varphi_1(t)\, dt \qquad v_2 = \int s(t)\, \varphi_2(t)\, dt}$$

4.2 Results

The computed signal space coordinates are:

- s_1 (t) → (v_1 , v_2) = (1.0000, 0.0000): s_1 lies entirely on the φ_1 axis, as expected since φ_1 was derived from s_1 .
- s_2 (t) → (v_1 , v_2) = (0.5000, 0.8660): The non-zero v_2 = 0.866 = √3/2 confirms that s_2 has a component orthogonal to s_1 .

- ***Figure 2: Signal Space Representation – s_1 (t) at (1, 0) and s_2 (t) at (0.5, 0.866)***



5. Parts 1.3& 1.4 – Effect of AWGN on Signal Space

5.1 Setup

The received signals are r_1 (t) = s_1 (t) + w(t) and r_2 (t) = s_2 (t) + w(t), where w(t) is zero-mean AWGN with variance σ^2. The signal energy is E = 1 J. The noise variance is set from the SNR ratio E/σ^2 in dB:

σ^2 = E / 10^(SNR_dB / 10)

For each of the 100 Monte Carlo samples, an independent noise vector of N = 1000 samples is generated and added to the signal. The noisy signal is then projected onto (φ_1 , φ_2) to obtain its signal space coordinates.

5.2 Results – E/σ^2 = −5 dB (High Noise)

At −5 dB, the noise variance is σ^2 = 3.16 J (much larger than E = 1 J). The scatter is very wide.

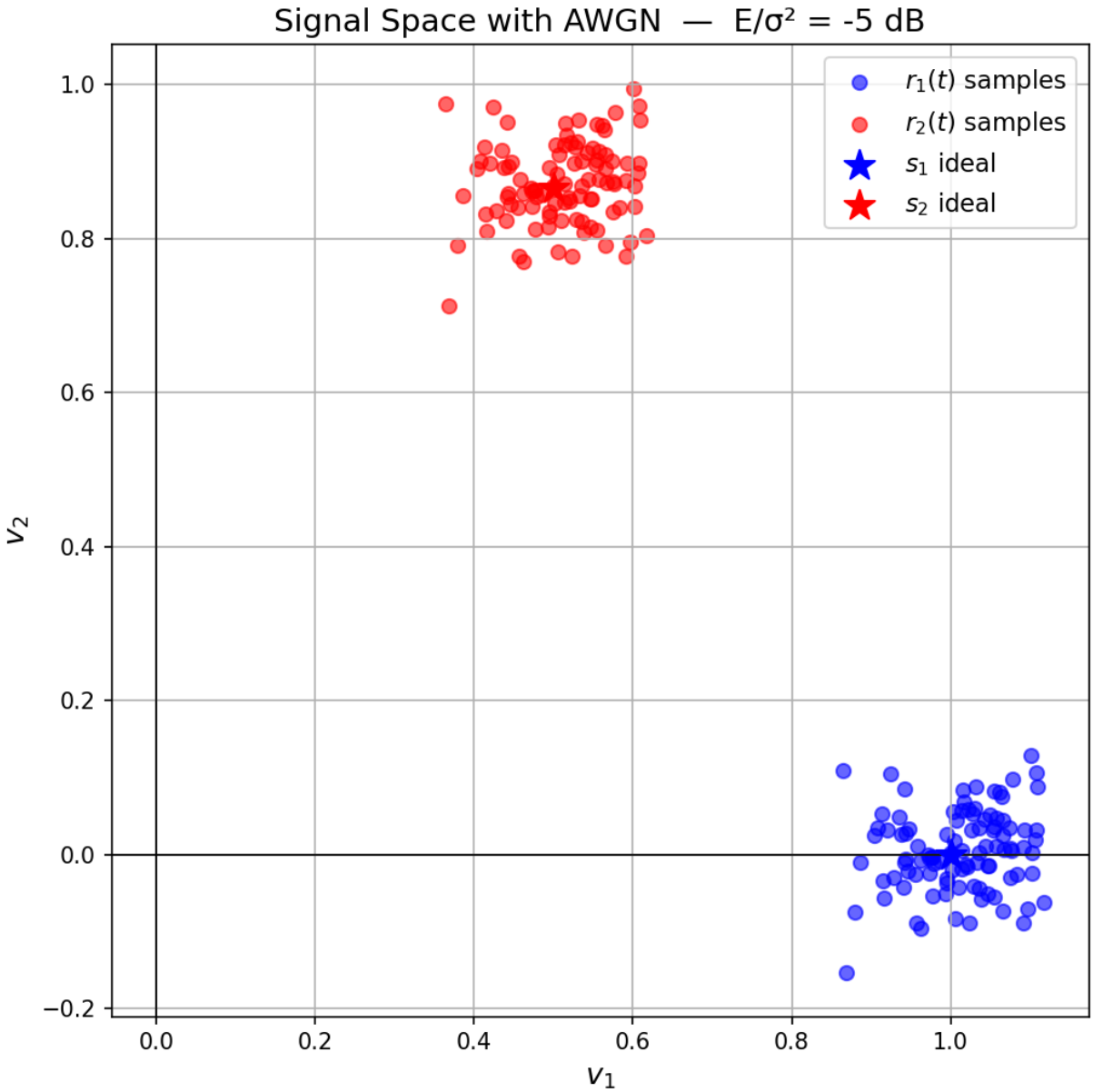

Signal Space with AWGN — E/σ² = -5 dB
$r_1(t)$ samples
$r_2(t)$ samples
s_1 ideal
s_2 ideal
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
v_2
0.0
0.2
0.4
0.6
0.8
1.0
v_1

Figure 3: Scatter Plot at E/σ² = −5 dB – Large scatter around ideal points

5.3 Results – E/σ² = 0 dB (Moderate Noise)

At 0 dB, σ² = E = 1 J. The two clusters are visible but still moderately spread.

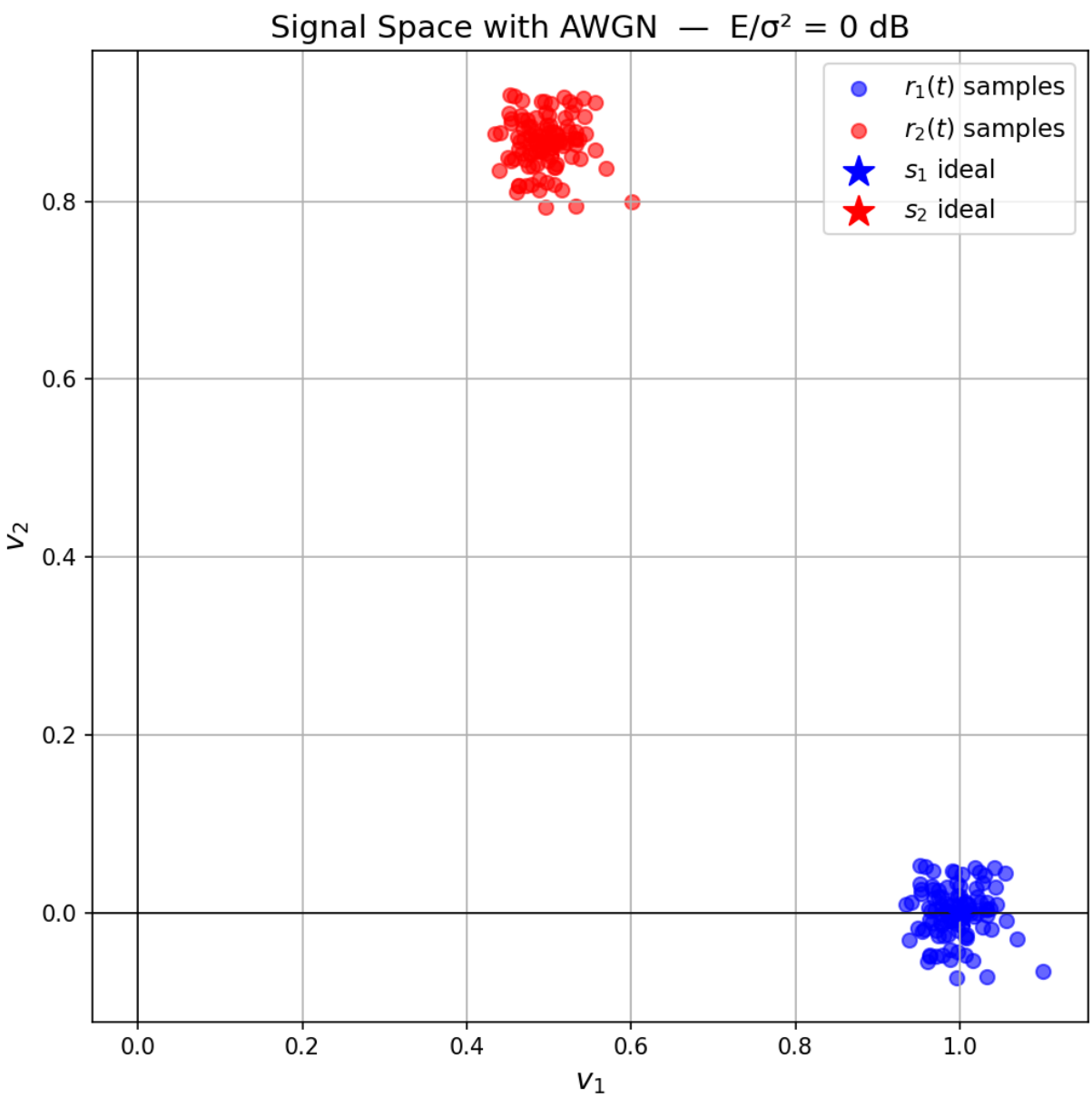


Figure 4: Scatter Plot at E/σ² = 0 dB – Moderate scatter, clusters clearly separated

5.4 Results – E/σ² = 10 dB (Low Noise)

At 10 dB, σ² = 0.1 J (much smaller than E). Points cluster tightly around ideal locations.

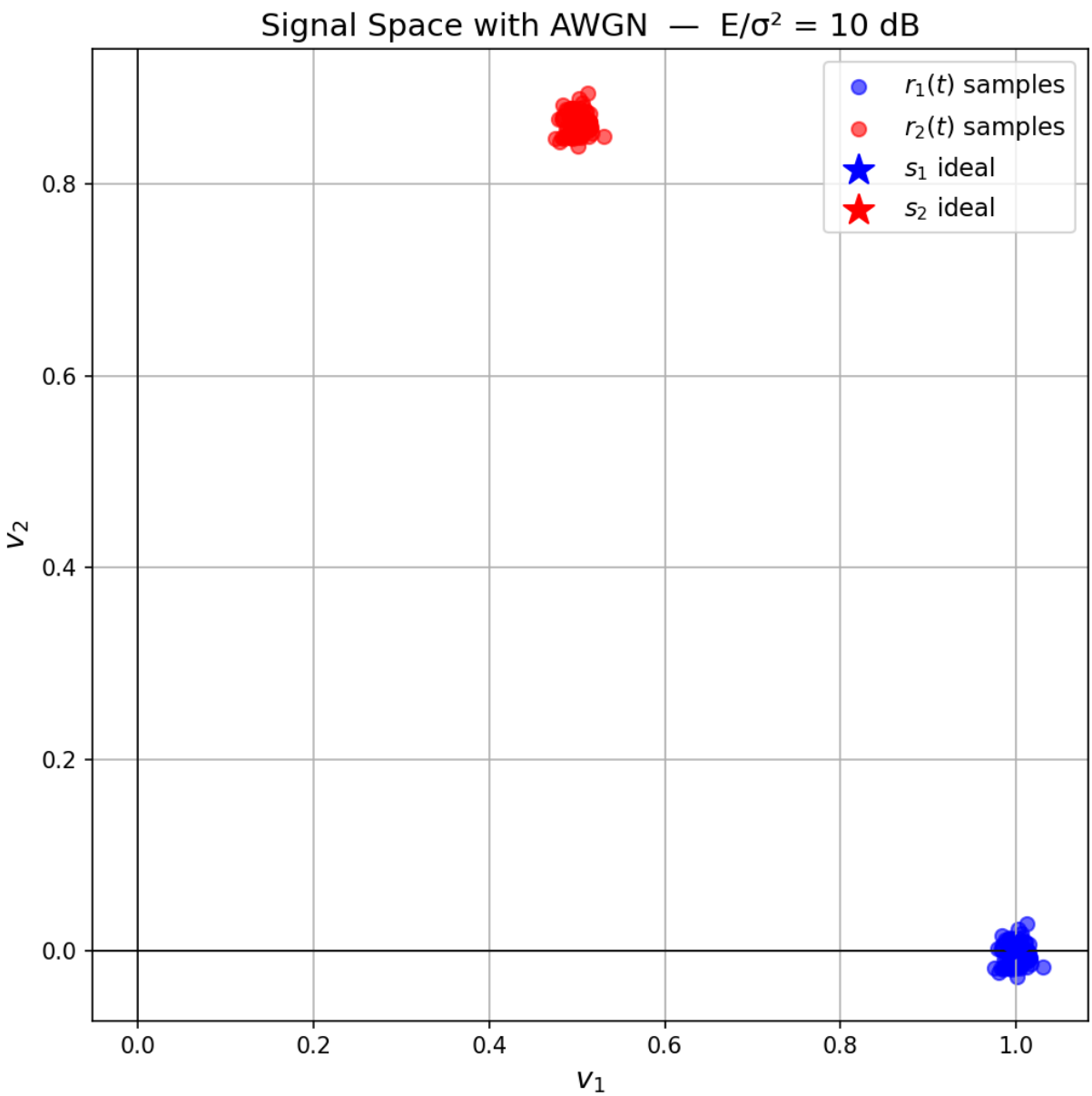


Figure 5: Scatter Plot at E/σ² = 10 dB – Tight clusters, low noise effect

6. Discussion – Requirement 4

Understand:

AWGN causes the received signal points to scatter randomly around the ideal noiseless signal points in the signal space. The noise adds a random displacement to each coordinate (v_1 , v_2) of the received signal.

The noise effect INCREASES as σ^2 increases (i.e., as E/σ^2 decreases):

- At $E/\sigma^2 = 10$ dB: $\sigma^2 = 0.1$ J → tight clusters; signals are easily distinguishable.
- At $E/\sigma^2 = 0$ dB: $\sigma^2 = 1.0$ J → moderate spread; clusters remain separated.
- At $E/\sigma^2 = -5$ dB: $\sigma^2 = 3.16$ J → wide scatter; clusters partially overlap, increasing the probability of detection error.

In conclusion, a higher SNR (E/σ^2) reduces the scatter in signal space, which lowers the probability of error in a decision-theoretic receiver.

Short-Answer:

AWGN randomly scatters signal points around ideal positions by shifting (v_1 , v_2).
Higher σ^2 (lower SNR) increases spread: tight → moderate → overlapping clusters.
Higher SNR reduces scatter, lowering detection error

7. Code Appendix

The following Python code implements all parts of the assignment.

```
import numpy as np
import matplotlib.pyplot as plt



# Signal definitions

N = 1000          # number of samples
T = 1.0           # signal duration (sec)
dt = T / N        # time step
t = np.linspace(0, T, N, endpoint=False)

# s1(t): rect pulse, amplitude 1 from 0 to 1
s1 = np.ones(N)

# s2(t): +1 from t=0.25 to 1, -1 from 0 to 0.25
s2 = np.where(t < 0.25, -1.0, 1.0)



# 1.1  Gram-Schmidt Orthogonalization

def GM_Bases(s1, s2, dt):
    """
    Returns orthonormal basis functions phi1, phi2
    using Gram-Schmidt for signals s1 and s2.
```

```
    dt  : time step (for numerical integration via sum*dt)
    """
    # --- phi1: normalize s1 ---
    energy1 = np.sum(s1 ** 2) * dt          # <s1, s1>
    phi1 = s1 / np.sqrt(energy1)

    # --- remove phi1 component from s2 ---
    proj = np.sum(s2 * phi1) * dt           # <s2, phi1>
    g2 = s2 - proj * phi1                   # residual

    # --- check if s2 is linearly independent of s1 ---
    energy_g2 = np.sum(g2 ** 2) * dt
    if energy_g2 < 1e-10:                   # linearly dependent
        phi2 = np.zeros(len(s1))
    else:
        phi2 = g2 / np.sqrt(energy_g2)

    return phi1, phi2



# 1.2  Signal Space Representation

def signal_space(s, phi1, phi2, dt):
    """
    Projects signal s onto phi1 and phi2.
    Returns scalar coordinates (v1, v2).
    """
    v1 = np.sum(s * phi1) * dt
    v2 = np.sum(s * phi2) * dt
    return v1, v2



# Requirement 1 — Compute & Plot Bases

phi1, phi2 = GM_Bases(s1, s2, dt)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
```

```
for ax, phi, label in zip(axes, [phi1, phi2], [r'$\phi_1(t)$', r'$\phi_2(t)$']):
    ax.plot(t, phi, linewidth=2)
    ax.set_title(label, fontsize=14)
    ax.set_xlabel('Time (sec)', fontsize=12)
    ax.set_ylabel('Amplitude', fontsize=12)
    ax.legend([label], fontsize=11)
    ax.grid(True)
fig.suptitle('Gram-Schmidt Orthonormal Basis Functions', fontsize=15)
plt.tight_layout()
plt.savefig('fig1_bases.png', dpi=150)
plt.show()



# Requirement 2 — Signal Space of s1, s2

v1_s1, v2_s1 = signal_space(s1, phi1, phi2, dt)
v1_s2, v2_s2 = signal_space(s2, phi1, phi2, dt)

print(f"s1 coordinates: ({v1_s1:.4f}, {v2_s1:.4f})")
print(f"s2 coordinates: ({v1_s2:.4f}, {v2_s2:.4f})")

plt.figure(figsize=(6, 6))
plt.scatter([v1_s1], [v2_s1], s=150, color='blue',  zorder=5, label=r'$s_1(t)$')
plt.scatter([v1_s2], [v2_s2], s=150, color='red',   zorder=5, label=r'$s_2(t)$')
plt.axhline(0, color='k', linewidth=0.8)
plt.axvline(0, color='k', linewidth=0.8)
plt.xlabel(r'$v_1$ (projection onto $\phi_1$)', fontsize=12)
plt.ylabel(r'$v_2$ (projection onto $\phi_2$)', fontsize=12)
plt.title('Signal Space Representation of $s_1$ and $s_2$', fontsize=14)
plt.legend(fontsize=12)
plt.grid(True)
plt.tight_layout()
plt.savefig('fig2_signal_space.png', dpi=150)
plt.show()



# Requirement 3 — AWGN Scatter Plots
```

```
n_samples = 100

# Energy of s1 (s1 and s2 have the same energy here)
E = np.sum(s1 ** 2) * dt
print(f"\nSignal energy E = {E:.4f} J")

snr_dB_list  = [-5, 0, 10]          # E/sigma^2 in dB
snr_lin_list = [10 ** (x / 10) for x in snr_dB_list]

for snr_dB, snr_lin in zip(snr_dB_list, snr_lin_list):
    sigma2 = E / snr_lin             # noise variance
    sigma  = np.sqrt(sigma2)

    coords_r1 = []
    coords_r2 = []

    for _ in range(n_samples):
        # Generate a fresh noise vector for each sample
        w = np.random.normal(0, sigma, N)

        r1 = s1 + w
        r2 = s2 + w

        v1_r1, v2_r1 = signal_space(r1, phi1, phi2, dt)
        v1_r2, v2_r2 = signal_space(r2, phi1, phi2, dt)

        coords_r1.append((v1_r1, v2_r1))
        coords_r2.append((v1_r2, v2_r2))

    coords_r1 = np.array(coords_r1)
    coords_r2 = np.array(coords_r2)

    plt.figure(figsize=(7, 7))

    # Scatter: noisy received signals
    plt.scatter(coords_r1[:, 0], coords_r1[:, 1],
                s=40, color='blue', alpha=0.6, label=r'$r_1(t)$ samples')
    plt.scatter(coords_r2[:, 0], coords_r2[:, 1],
```

```
            s=40, color='red',  alpha=0.6, label=r'$r_2(t)$ samples')

    # Mark ideal signal points
    plt.scatter([v1_s1], [v2_s1], s=200, color='blue',
            marker='*', zorder=5, label=r'$s_1$ ideal')
    plt.scatter([v1_s2], [v2_s2], s=200, color='red',
            marker='*', zorder=5, label=r'$s_2$ ideal')

    plt.axhline(0, color='k', linewidth=0.8)
    plt.axvline(0, color='k', linewidth=0.8)
    plt.xlabel(r'$v_1$', fontsize=13)
    plt.ylabel(r'$v_2$', fontsize=13)
    plt.title(f'Signal Space with AWGN  —  E/σ² = {snr_dB} dB', fontsize=14)
    plt.legend(fontsize=11)
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(f'fig3_scatter_{snr_dB}dB.png', dpi=150)
    plt.show()



# Requirement 4 — Answer (printed)

print("""
Requirement 4 — Effect of noise on signal space:
- AWGN causes the received signal points to scatter around the ideal signal points.
- The scatter INCREASES as sigma^2 INCREASES (i.e. lower E/sigma^2 ratio).
- At E/sigma^2 = 10 dB  → points cluster tightly near s1 and s2 (low noise).
- At E/sigma^2 = -5 dB  → points are widely spread (high noise), making it
  harder to distinguish between s1 and s2.
""")
```